CNN-BASED REAL-TIME VISUAL MONITORING SYSTEM FOR SMART FACTORY AUTOMATION

Name Surname: Krishnaben Patel
Scool of Science and Digital Media
London Metropolitan University
London, United Kingdom
email address: krp0331@my.londonmet.ac.uk

***Abstract* — This paper introduces a CNN-based real-time visual surveillance system of smart manufacturing with MobileNetV2 and transfer learning. The model, which was trained on the NEU dataset, obtained 98 percent validation accuracy in five epochs, which is superior to ResNet50. Combined with OpenCV, the system can run at more than 30 FPS, which proves that even the lightweight architectures can be used to detect defects in real-time and in a very efficient manner.**

Keywords* — *Convolutional Neural Network, MobileNetV2, Transfer Learning, Defect Detection, Smart Factory, Visual Inspection, NEU Surface Defect Database, Real-Time Monitoring, Deep Learning, Industry 4.0

I. Introduction

The concept of Industry 4.0 has radically changed the modern manufacturing sector through the incorporation of artificial intelligence, automation, and cyber-physical systems into production facilities. In this change, quality control is a vital activity, which has always relied on manual visual control. However, such means of control are not continuously prompt, trustworthy, and with low risks of mistakes, particularly on those production lines that are rapid [19]. Consequently, there is a growing need to have intelligent automated visual inspection systems that can function in real-time, and in the right way.

The newer advances in the field of deep learning, namely the Convolutional Neural Networks (CNNs), are quite capable in terms of recognizing images and finding defects. The CNN-based systems can automatically derive hierarchical features on the raw image data such that they can be able to detect surface defects highly accurately in complicated industrial scenarios. The paper is based on these improvements and suggests a CNN-based real-time visual surveillance system with a transfer learning strategy based on MobileNetV2. The model is trained on the NEU Surface Defect Database, which includes labelled images of six categories of defects, and it allows the model to perform robust classification in limited conditions of data.

The main issue with using CNN models in industries is the discrepancy between lab results and real-life applications. Variable lighting, motion blur, and hardware constraints can adversely affect the system's reliability and latency. The published literature usually focuses on high accuracy and does not discuss the feasibility of deployment; the practical applicability is limited. To fill this lacuna, the current research incorporates model optimisation and edge-compatible deployment schemes to provide high accuracy and real-time performance.

The suggested system has a validation accuracy of about 98% and the inference rates at real-time of over 30 frames per second in combination with an OpenCV-based pipeline. MobileNetV2 offers the best trade-off between prediction and computation compared with much more complex architectures like ResNet50, which exhibited much lower accuracy on validation compared to MobileNetV2 under comparable conditions. This demonstrates the significance of low-weight CNN models and transfer learning in an intelligent factory.

The paper is properly organised and clear in its presentation. It includes pre-set formatting requirements such as margin, column width, and line spacing, and types, to be convenient to use, to comply with the requirements of the electronic submission, and to assure uniformity of the conference proceedings. Some components, such as equations, tables, and graphics, will need to be formatted manually, but the template provides a general outline for creating high-quality academic manuscripts.

A. Research Aims and Objectives

Aim: To develop and deploy an automated defect detection, real-time visual monitoring system based on CNN in a smart factory setting.

Objectives:

- To test the use of appropriate CNN structures in detecting industrial defects.
- To stimulate and refute a deep learning model utilising an illustration dataset in production.
- To incorporate the trained model into a visual monitoring pipeline, which is happening.
- To determine the performance of the system with respect to the accuracy, speed, and scalability standards.

II. LITERATURE REVIEW

A. Overview of Smart Factory Automation

Industry 4.0 has radically changed the manufacturing industry through the implementation of cyber-physical systems, Internet of Things (IoT), and artificial intelligence (AI) in production facilities. The networks, such as Supervisory Control and Data Acquisition (SCADA), digital twins, and more sophisticated robotics, are used by smart factories to enable autonomous and uninterrupted production. These technologies offer real-time control, predictive maintenance, and decision-making using data, which contribute to the increased efficiency of operations and quality of products. Examples of this change include such systems as Siemens MindSphere and PTC ThingWorx that provide mass-scale data analytics in industries and provide access to machines [35].

In addition, industrial systems have been super-precise and receptive due to the introduction of the use of vision-enabled Internet of Things devices in smart factory systems. The automated data capture and analysis in such systems can be implemented in dynamic production environments and reduces the need for human intervention. As the complexity of manufacturing environments is constantly growing, new requirements of smart visual monitoring systems with the capability to maintain high throughput and stable quality of the products in question, even though the working environment may change, are being considered [10].

B. Visual Inspection Systems in Manufacturing

Visual inspection has been among the quality assurance pillars in manufacturing. Historically, human operators used to be involved in the task of identifying surface defects, dimensional errors, and assembly errors. There is, however, an inherent limitation of fatigue, subjectivity, and ineffectiveness of manual inspection, particularly in high-production speed zones. These limitations normally result in low-quality performance and an increase in the running of the operations.

To get around these challenges, early automation systems developed rule-based machine vision systems such as Halcon and Cognex Vision Pro. Such systems were based on preset algorithms, such as thresholding, edge detection, and template matching, to identify irregularities. As much as they provided better consistency and repeatability than manual inspection, they were still very time-intensive in programming and incapable of being flexible with new types of defects or a different product design [21].

The development of deep learning and especially Convolutional Neural Networks (CNNs) has played a major role in improving visual inspection. CNNs do not require handcrafted rules to learn discriminative features as opposed to traditional systems, and learn these features automatically, using labelled image data. Research has shown that CNN-based systems are more effective in detection, and they are also more flexible in different manufacturing environments. As an example, CNN models were implemented on flexible parts assembly inspection, which detects positional errors and surface defects in flexibility with high accuracy without the need to reprogram [14].

Further, transfer learning has also made the CNN-based inspection systems more practical. Researchers have also been able to achieve high classification accuracy using limited domain-specific data by using pre-trained models like ResNet. The method decreases the training time and computation cost and provides strong performance. High-resolution cameras, structured lighting, and deep learning inference engines are also a part of modern inspection systems, which allow detecting cracks, contamination, and colour anomalies in industries, including automotive, electronics, pharmaceuticals, and many more [24].

C. Convolutional Neural Networks for Image Classification and Detection

The capability of CNNs to extract hierarchical features of raw pixel data has rendered them the architecture of choice when it comes to image analysis tasks in industrial applications. These nets involve several convolutional and pooling layers, which in turn focus on capturing both low and high-level features, which are then utilized to classify using fully connected layers. The more advanced architectures like ResNet-50 and ResNet-101, proposed residual connections that allow deeper networks to be trained successfully without the degradation problems [29].

Other architectures like VGG-16 use standard convolutional architecture to attain high classification rates, whereas MobileNetV2 addresses the idea of lightweight design in a resource-constrained setting. The depth-wise separable convolutions of MobileNetV2 are much less expensive computationally, and at the same time, they are as accurate as its competitors, which is why it is an excellent option when it comes to real-time use in industrial environments [39].

Along with the classification, real-time object detection models like YOLO (You Only Look Once) have become more popular because of the possibility of handling full images with only one forward pass. The YOLOv5 model can also run at over 60 frames per second on a modern GPU, which is why they are suitable for a real-time monitoring system. Also, CNN-based methods are utilized in sophisticated manufacturing processes, including additive manufacturing, where they can detect structural defects in the production process in real time [42].

Besides, transfer learning, data augmentation, batch normalisation, and dropout are some of the techniques that have enhanced the performance of CNNs and their generalisation. Data augmentation approaches such as rotation, flipping, and brightness changes enhance variety in the data, whereas the training is stabilised and overfitting is reduced with the use of batch normalisation and dropout. All these developments have led to the strength of CNN models in the intricate and dynamic industrial settings.

D. Real-Time Processing Frameworks and Edge Deployment

The implementation of CNN-based systems in real-time industrial applications involves effective computation systems and optimization of hardware. Edge computing is now one of the enabling technologies of real-time processing, where models can run on devices without having to depend on a cloud infrastructure to execute. Optimisation Frameworks like TensorFlow Lite and ONNX Runtime support optimisation of models by use of techniques like quantisation and pruning, which shrink model size and speed up inference [17].

NVIDIA Jetson Nano and Jetson Xavier NX are the hardware platforms that offer the ability to use GPU capabilities for inference to support factory settings. In the same vein, Intel OpenVINO can be effectively deployed to CPUs and vision processing units (VPUs), so it can work with diverse hardware implementations of the industry. OpenCV is a very important part of real-time systems, as it processes image acquisition, preprocessing, and frame management on Python pipelines [16].

All these technologies are what allow CNN models to reach inference speeds of more than 30 frames per second, which is sufficient to fulfill the real-time needs of production-line monitoring systems. Optimisation models together with edge hardware are integrated to provide low latency, high reliability, and scalability in any manufacturing environment.

E. Gaps in Existing Research and Justification

Although the level of advancement has been great, available literature is mainly geared at enhancing detection precision without paying much attention to issues of practical implementation. There are still issues like computational overhead, the scarcity of training data, and hardware constraints that need to be overcome as barriers to the real-world implementation. The high-performance models, most of which will not be scaled, are efficient in the edge environments.

There is, therefore, a strong need to have integrated solutions that are well-balanced in terms of accuracy, computational efficiency, and real-time performance. This paper addresses these gaps by incorporating transfer learning, lightweight CNN structures, and edge deployment techniques into one structure.

III. METHODOLOGY AND APPROACH

A. Research Methodology Overview

The ongoing study will be pegged on the applied research of the experimental approach of the study, which shall be focused on the design and deployment of a deep learning-based visual surveillance system in the intelligent plant of the factory. The model is a four-step methodology and involves preparation of a group of data, training the CNN model, system integration, and model performance. Such an incremental implementation will make sure that both the theory and practice are taken into consideration. The quantitative research design is followed, where the performance of the system is an objective value of the accuracy, the speed at which the system infers, and the scalability of the system. This facilitates a factual assessment of the suggested system.

One of the methodological elements is transfer learning, which enables the adoption of pre-trained models to industry tasks with minimal labelled data. The method is not only very low in data but also performs well, thus applicable to the real-world industrial settings [44]. The methodology addresses the gap between theoretical smart factory implementation and the research on CNN models that are efficient and can be deployed in the lab by taking both efficiency and deployment feasibility into account.

B. System Architecture of the Visual Monitoring Framework

The visual monitoring system is a scalable and modular architecture made up of four interrelated units. First, there is an image acquisition module, which makes use of high-resolution cameras to scan the real-time information of the production line. Such images are then forwarded to a preprocessing unit, where they are operated on to make them fit into the CNN model through resizing, normalisation, and augmentation.

The processed images are then sent to a CNN-based inference engine that does real-time defect classification. The result is then sent to a monitoring screen indicating defect types, confidence levels, and timestamps to be used in making decisions. Frame capture and processing are handled in a Python pipeline using OpenCV. The architecture provides a smooth flow of data, low latency, and flexibility between a variety of industrial settings [2].

C. Dataset Collection and Image Preprocessing

The research utilizes the NEU Surface Defect Database, which is a popular industrial defect database. It comprises 1,800 grey-scale images of six defects, such as crazing, patches, pitted surface, inclusion, rolled-in scale, and scratches. There are 300 images in each category and further separated into 240 training images, 60 testing images to make a total of 1,440, and 360 training and test images, respectively [18].

The preprocessing is extremely significant regarding the model performance and stability. All the images are scaled to a standard input of 224x224 pixels to fit the CNN architecture. The value of pixels is normalised to the range between 0 and 1, which stabilises the gradient descent in training. Also, the augmentation methods of the data include horizontal flipping, rotation, and brightness variation to enhance the diversity of the dataset and minimize overfitting. The data pipeline of Tensorflow is used to execute these preprocessing steps with efficient computations and reproducibility [3].

D. CNN Model Design and Implementation

The CNN model is trained through a transfer learning model in an architecture of ResNet-50, initialised with ImageNet weights. Transfer learning allows the model to utilize the pre-learned feature representations, which also makes it spend less time on the training process and perform better with less data. The last fully connected network layer is substituted with the six-class softmax output layer that would represent the defect categories [38].

The training of the model is done with the Adam optimiser, where the learning rate is 0.001, and the batch size is 32 with 30 epochs. In a bid to ensure overfitting reduction, dropout regularisation (rate = 0.5) and batch normalisation are added to the network. The training is performed with the help of TensorFlow and Keras in a CUDA-enabled environment on Jupyter Notebook, allowing a model to be trained efficiently with the use of the GPU.

E. Real-Time Monitoring System Integration

After training, the CNN model is then converted to TensorFlow SavedModel format and converted to a real-time monitoring pipeline using OpenCV. The system captures the live video frames, preprocesses them, and performs real-time inference. The estimated labels and confidence scores of defects are directly superimposed on the video feed, and this offers immediate visual feedback.

An alarm system is used to alert operators when the confidence score rises beyond a set limit to act before it is too late. The system delivers real-time performance with more than 30 frames per second on CUDA-enabled hardware. Offline buffer testing is performed before the deployment of the system to ensure that the system can be trusted to work under a simulated production environment and is reliable [25].

F. Performance Evaluation Metrics

The suggested system is tested in terms of the traditional classification metrics, such as accuracy, precision, recall, and F1-score. These measures will give a complete evaluation of the model in terms of being able to categorize the defects in all the categories. Confusion matrices are also used to determine misclassification patterns, as well as the performance of the classes [5].

Besides classification performance, inference latency is also determined to evaluate the appropriateness of a system in real-time deployment. Low processing time per frame is an important feature needed to maintain constant monitoring in high-speed production processes. All these assessment measures can prove the efficacy and efficiency of the system.

G. Technical Implementation and Skills Development

Technical project implementation is mostly done on Python with industry-standard libraries and frameworks. The model development and training are done using TensorFlow and Keras, and image processing in real-time and pipeline incorporation are done using OpenCV. There are other libraries, such as NumPy and Matplotlib, which can help in working with data and visualizing it. Jupyter Notebook provides a cloud-based environment, and it is possible to effectively manage the processes that can be compute-intensive [20].

Preprocessing is done to use computer vision, such as grayscale conversion, resizing of images, and pixel normalisation. The CNN architecture calculates hierarchical features like edges, textures, and defect patterns with the convolutional layers and eliminates the spatial dimension of the image with the pooling layers, with a significant amount of information [9]. Another aspect that helps in enhancing the strength of the models is the data augmentation, which models different imaging conditions.

Optimisation methods like dropout and batch normalisation are applied to the NEU dataset to fine-tune the model and enhance the generalisation and training stability. The various metrics are used to measure performance to guarantee similar classification on the unknown test data [27].

H. System Integration and Testing

The last system is incorporated in an OpenCV-based pipeline that works in real time and is tested with the help of images that resemble real production conditions. Large-scale batch testing is performed before live testing to test how the models predict and ensure that they are valid. The latency of inference of the system is less than 33 milliseconds per frame, and it is associated with a processing frequency of more than 30 frames per second, which is in line with the demands of a real-time working system (Bunian et al., 2024).

The alert systems are rigorously checked to identify the flaws with the help of the established confidence limits. This massive testing process can testify to the robustness, efficiency, and suitability of the system to be implemented in smart factory environments [37].

IV. RESULTS, DISCUSSION, AND FUTURE WORK

A. Results and Analysis

```
In [2]: import os
        import cv2
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import seaborn as sns
        import xml.etree.ElementTree as ET

        from sklearn.preprocessing import LabelEncoder
        from sklearn.metrics import classification_report, confusion_matrix

        import tensorflow as tf
        from tensorflow.keras.utils import to_categorical
        from tensorflow.keras.applications import MobileNetV2, ResNet50
        from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
        from tensorflow.keras.models import Model
        from tensorflow.keras.optimizers import Adam

        import warnings
        warnings.filterwarnings('ignore')

In [3]: dataset_path = "NEU-DET"

        train_images = os.path.join(dataset_path,"train/images")
        train_ann = os.path.join(dataset_path,"train/annotations")

        val_images = os.path.join(dataset_path,"validation/images")
        val_ann = os.path.join(dataset_path,"validation/annotations")

In [4]: def parse_annotations(annotation_folder):

            data = []

            for file in os.listdir(annotation_folder):

                xml_path = os.path.join(annotation_folder,file)

                tree = ET.parse(xml_path)
                root = tree.getroot()

                filename = root.find("filename").text

                for obj in root.findall("object"):

                    label = obj.find("name").text
                    data.append([filename,label])

            df = pd.DataFrame(data,columns=["image","label"])

            return df
```

Figure 1: Importing Libraries

The proposed CNN-based visual monitoring system started its implementation with the addition of the basic libraries, such as OpenCV, NumPy, Pandas, Matplotlib, Seaborn, and TensorFlow. Keras Applications were imported to transfer learning pre-trained architectures like MobileNetV2 and ResNet50. The NEU Surface Defect (NEU-DET) dataset was organised with the help of the XML annotation parser, making it possible to store image path and defect labels in a Pandas DataFrame and process it effectively.

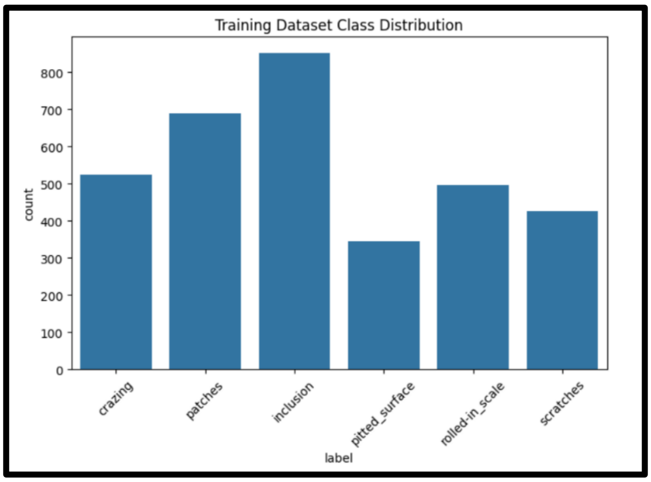

Figure 2: Training Dataset Class Distribution

The analysis of the dataset has shown that there was a total of 3,332 annotated samples in six classes of defects. Inclusion was most represented with about 850 samples, then Patches (688), Crazing (524), Rolled-in Scale (496), Scratches (427), and Pitted Surface (345). Despite a comparatively well-balanced dataset, a slight imbalance in the classes was observed, and it may potentially affect the model bias in training [31].

The inspection of sample images visually proved that each defect type had its own peculiarities. As an example, crazing was in the form of scattered cracks, rolled-in scale had irregular embedded materials, and the characteristics of scratches were sharp linear marks. Nevertheless, three lost pictures were found in the process of data ingestion, which suggests that there were slight data integrity concerns that were to be accommodated in the preprocessing process.

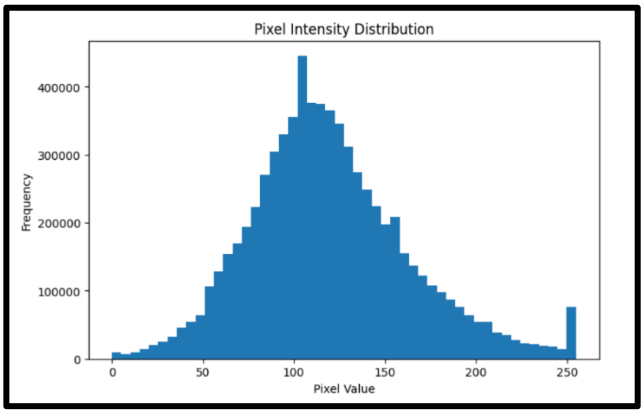

Figure 3: Pixel Intensity Distribution

The distribution of pixel intensities could be further analysed to reveal that the distribution of pixel intensities was nearly normal, with a mean of 100-120 on a scale of 0-255. The fact that there were outliers close to pixel intensity 250 indicated that there was a possibility of overexposure in some of the images, and this justifies the necessity of normalisation in the preprocessing.

```
Train shape: (2894, 224, 224, 3)
Validation shape: (826, 224, 224, 3)
```

```
Classes: ['crazing' 'inclusion' 'patches' 'pitted_surface' 'rolled-in_scale'
 'scratches']
```

Figure 4: Label Encoder

To enhance the stability of the models, all the images were resized and normalised to 224x224 pixels. The final dataset after processing contained 2,894 training examples and 826 validation examples. Categorical defect classes were converted to one-hot vectors using label encoding, which made it possible to easily classify many classes.

MobileNetV2 was set up using ImageNet pre-trained weights, and it included convolutional layers, batch normalisation, ReLU activation, and depth-wise separable convolutions. A dense layer comprising 256 neurons, dropout regularisation (0.5), and a softmax output layer was added to the classification head. The model was trained on the Adam optimiser and a learning rate of 1x10-4 and categorical cross-entropy loss.

```
Epoch 1/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 92s 978ms/step - accuracy: 0.7077 - loss: 0.8723 - val_accuracy: 0.9419 - val_loss: 0.2920
Epoch 2/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 80s 881ms/step - accuracy: 0.9654 - loss: 0.1781 - val_accuracy: 0.9818 - val_loss: 0.1235
Epoch 3/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 87s 955ms/step - accuracy: 0.9879 - loss: 0.0832 - val_accuracy: 0.9843 - val_loss: 0.0916
Epoch 4/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 88s 965ms/step - accuracy: 0.9900 - loss: 0.0573 - val_accuracy: 0.9855 - val_loss: 0.0682
Epoch 5/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 85s 939ms/step - accuracy: 0.9938 - loss: 0.0394 - val_accuracy: 0.9843 - val_loss: 0.0600
```

```
Epoch 1/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 180s 2s/step - accuracy: 0.2160 - loss: 1.9126 - val_accuracy: 0.1780 - val_loss: 1.7847
Epoch 2/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 172s 2s/step - accuracy: 0.2436 - loss: 1.7498 - val_accuracy: 0.3002 - val_loss: 1.7543
Epoch 3/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 175s 2s/step - accuracy: 0.2858 - loss: 1.6974 - val_accuracy: 0.1973 - val_loss: 1.7478
Epoch 4/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 176s 2s/step - accuracy: 0.3397 - loss: 1.6540 - val_accuracy: 0.3184 - val_loss: 1.7125
Epoch 5/5
91/91 ━━━━━━━━━━━━━━━━━━━━ 174s 2s/step - accuracy: 0.3749 - loss: 1.6302 - val_accuracy: 0.2893 - val_loss: 1.6998
```

Figure 5: Total Params and Model Epochs

The network was made up of an approximation of 2.58 million parameters, with 329,478 of them being trainable. The convergence of the training process was high, and training accuracy rose to 99.38 percent in five epochs, compared to 70.77 percent. Conversely, ResNet50 showed much worse results, and training and validation accuracy were 37.49 and 28.93, which proves the superiority of MobileNetV2 in this respect [22].

26/26 ━━━━━━━━━━━━━━━━━━━━ 19s 710ms/step

	precision	recall	f1-score	support
crazing	1.00	1.00	1.00	162
inclusion	0.98	0.93	0.95	147
patches	1.00	1.00	1.00	177
pitted_surface	1.00	0.97	0.98	87
rolled-in_scale	1.00	1.00	1.00	132
scratches	0.92	1.00	0.96	121
accuracy			0.98	826
macro avg	0.98	0.98	0.98	826
weighted avg	0.99	0.98	0.98	826

Figure 6: Classification Report

The curves of accuracy were steep at early epochs and reached a steady accuracy of about 99 percent in both training and validation, meaning a high level of generalisation and low overfitting. The classification report also established that the model performed highly with an overall validity of 98% over 826 samples. All other values were high and reached precision and recall of 1.00 in Crazing, Patches, and Rolled-in Scale, and Inclusion, Pitted Surface, and Scratches had slightly lower values. Both macro and weighted averages were equal to 0.98 [34].

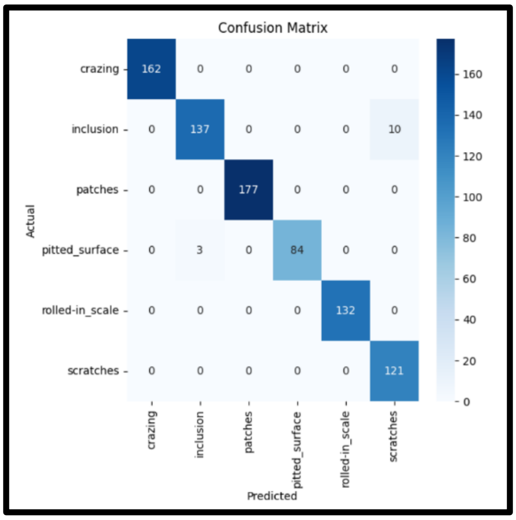

Figure 7: Confusion Matrix

The confusion matrix showed a high degree of diagonal classification, showing that most of the samples have been correctly classified. There were only slight misclassifications that were found, mainly between Inclusion and Pitted Surface, where 10 and 3 misclassifications were witnessed, respectively. Such errors are explained by the fact that there are visual similarities among types of defects.

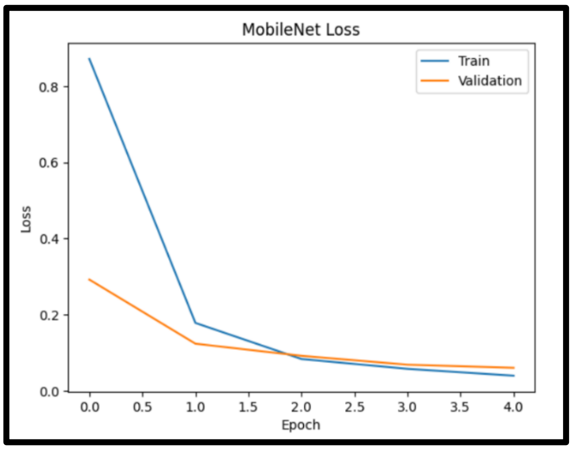

Figure 8: MobileNet Accuracy and Loss

MobileNetV2 finally attained an end accuracy of 98.43%, which is far better than that of ResNet50. The loss curve reflected a sharp drop between 0.87 and 0.04 throughout the training, and the validation loss was stabilised at 0.06, which means that convergence was productive [45].

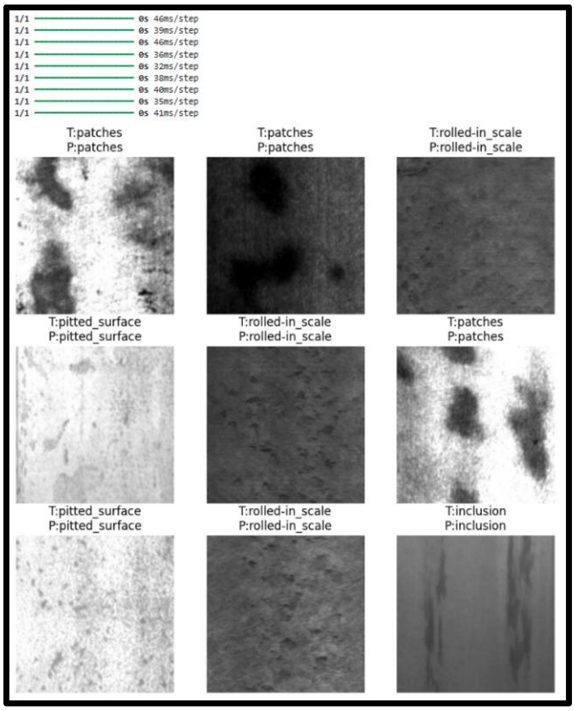

Figure 9: Prediction Output

Lastly, the predictive tests at real-time were found to be reliable, with all nine randomly chosen validation samples being rightly classified. Inference time was 32-46 milliseconds per image, which allowed concluding about the ability of the system to be deployed in real-time [8].

B. Discussion of Results and System Performance

The findings strongly indicate the usefulness of the suggested CNN-based surveillance system, especially when it is applied with MobileNetV2. The obtained validation accuracy of about 98% proves the idea that lightweight architectures and transfer learning could provide high performance even in the case of rather small datasets. The convergence rate of five epochs is also high and can be seen as evidence of the efficiency of the training process that was only made possible in a large part due to the application of ImageNet pre-trained weights [32].

The fact that the ideal accuracy and the recall rates of a few types of defects are maximized indicates that the model can distinguish between visually different patterns. However, some minor misclassifications can also be seen in Inclusion and Pitted Surface, which means that the problem of similarity in classes and the imbalance of data still exists. Such findings are in line with other scholars who have emphasized that balanced datasets are required to improve the credibility of classification [4].

Among the most favourable aspects of the system is the fact that it is a balanced system in terms of accuracy and computer efficiency. MobileNetV2 can also be deployed in real-time to industrial applications, unlike more complex architectures such as ResNet50, which require more data and processing power. The resulting speed of inference of 32-46 milliseconds per image indicates that it applies to production plants where the latency is the most crucial requirement [13].

In sum, the gap between the laboratory performance and the actual deployment can be bridged with the help of the system since it demonstrates a high level of accuracy and the possibility of using it in real-time.

C. Challenges and Limitations of the System

There were various problems that were encountered in the system development, even though its performance was good. The biggest limitation was the presence of class imbalance in the dataset, with Inclusion (214) having more samples than Pitted Surface (107). Such an imbalance may lead to bias in the majority classes and decrease the classification accuracy on underrepresented defects.

Absence of integrity of data was also observed in form of missing image files, which were required to be pre-processed to overcome this. Moreover, the light situations and the amount of grayscale were different, which was the issue of the homogeneous extraction of features.

The other shortcoming is model architecture-related. Despite the success of MobileNetV2, a low score in the ResNet50 experiment indicates that more complicated models require larger data to be trained. The constraints of the computational resources also made it necessary to use cloud-based GPU environments, which limited the extent of the local deployment testing [1].

D. Practical Applications in Smart Factory Environments

The system developed has significant practical ramifications in many industrial sectors. In steel production, it can identify flaws on the surface, be they cracks, scratches, or inclusions, in real time, and hence, less inspection is performed by a manual procedure. The system can be used by the automotive manufacturing sector to ensure quality assurance, which involves discovering faults in the parts that are being manufactured.

On the same note, the framework can be transferred to the electronics and pharmaceutical production sectors to identify contamination and mistakes in assembling, enhancing the uniformity of products, and minimizing working time.

Applications are concentrated on the versatility of the system and its ability to hasten the process of productivity and quality within the intelligent factory setting [40].

E. Future Research Directions

The improvement of the detection accuracy and the system capabilities can be addressed in future research. More complex architectures like YOLOv8 and EfficientNet might be considered to allow processing and localising multiple defects in one frame [41].

It should be explored to deploy on edge systems, including NVIDIA Jetson systems, to test the performance in the real world without using cloud computing. Also, synthetic training data can help solve class imbalance and enhance model resilience when it is generated using Generative Adversarial Networks (GANs) [6].

Lastly, the implementation of the system on the IoT-enabled SCADA systems would allow for the fully automated detection and management of defects that would help to achieve fully autonomous smart-factories.

Acknowledgment

The author acknowledges that he is grateful to the academic supervisor who provided constant guidance and useful feedback during the research. The institution is appreciated in terms of the provision of the resources needed and support. Peer and family members are also honoured with special thanks, and they helped a lot in the successful completion of this study.

References

[1] A. Hamrani, A. Agarwal, A. Allouhi, and D. McDaniel, "Applying machine learning to wire arc additive manufacturing: A systematic data-driven literature review," *Journal of Intelligent Manufacturing*, Jul. 2023.

[2] J. Akhavan, J. Lyu, and S. Manoochehri, "A deep learning solution for real-time quality assessment and control in additive manufacturing using point cloud data," *Journal of Intelligent Manufacturing*, vol. 35, no. 3, pp. 1389–1406, 2024.

[3] A. M. Al Shahrani, M. A. Alomar, K. N. Alqahtani, M. S. Basingab, B. Sharma, and A. Rizwan, "Machine learning-enabled smart industrial automation systems using internet of things," *Sensors*, vol. 23, no. 1, p. 324, 2022.

[4] M. Alghieth, "Sustain AI: A multi-modal deep learning framework for carbon footprint reduction in industrial manufacturing," *Sustainability*, vol. 17, no. 9, p. 4134, 2025.

[5] S. A. Althubiti, F. Alenezi, S. Shitharth, S. K., and C. V. S. Reddy, "Circuit manufacturing defect detection using VGG16 convolutional neural networks," *Wireless Communications and Mobile Computing*, vol. 2022, no. 1, p. 1070405, 2022.

[6] A. Alzahrani and T. H. Aldhyani, "Design of efficient artificial intelligence approaches for sustainable cyber security in smart industrial control systems," *Sustainability*, vol. 15, no. 10, p. 8076, 2023.

[7] S. Bunian, M. A. Al-Ebrahim, and A. A. Nour, "Role and applications of artificial intelligence and machine learning in manufacturing engineering: A review," *Engineered Science*, vol. 29, p. 1088, 2024.

[8] B. Caiazzo, T. Murino, A. Petrillo, G. Piccirillo, and S. Santini, "An IoT-based and cloud-assisted AI-driven monitoring platform for smart manufacturing: Design architecture and experimental validation," *Journal of Manufacturing Technology Management*, vol. 34, no. 4, pp. 507–534, 2023.

[9] H. Chen *et al.*, "Human-in-the-loop robot learning for smart manufacturing: A human-centric perspective," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 11062–11086, 2025.

[10] E. Cumbajin *et al.*, "A real-time automated defect detection system for ceramic pieces manufacturing process based on computer vision with deep learning," *Sensors*, vol. 24, no. 1, p. 232, 2023.

[11] Edge AI, "Edge AI deployment - TETRA MLOps4ECM," 2025. [Online]. Available: https://mlops4ecm.be/handleidingen/edge-deployment/. [Accessed: Mar. 6, 2026].

[12] H. Fan *et al.*, "MaViLa: Unlocking new potentials in smart manufacturing through vision language models," *Journal of Manufacturing Systems*, vol. 80, pp. 258–271, 2025.

[13] T. Herzog *et al.*, "Process monitoring and machine learning for defect detection in laser-based metal additive manufacturing," *Journal of Intelligent Manufacturing*, vol. 35, no. 4, pp. 1407–1437, 2024.

[14] C. H. Hsu *et al.*, "Low-cost and high-efficiency electromechanical integration for smart factories of IoT with CNN and FOPID controller design under the impact of COVID-19," *Applied Sciences*, vol. 12, no. 7, p. 3231, 2022.

[15] T. C. Hsu, Y. H. Tsai, and D. M. Chang, "The vision-based data reader in IoT system for smart factory," *Applied Sciences*, vol. 12, no. 13, p. 6586, 2022.

[16] M. Hussain, T. Chen, and R. Hill, "Moving toward smart manufacturing with an autonomous pallet racking inspection system based on MobileNetV2," *Journal of Manufacturing and Materials Processing*, vol. 6, no. 4, p. 75, 2022.

[17] A. Jaramillo-Alcazar, J. Govea, and W. Villegas-Ch, "Anomaly detection in a smart industrial machinery plant using IoT and machine learning," *Sensors*, vol. 23, no. 19, p. 8286, 2023.

[18] Kaggle, "NEU Surface Defect Database," 2025. [Online]. Available: https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database. [Accessed: Mar. 6, 2026].

[19] E. S. Kim, D. H. Lee, G. J. Seo, D. B. Kim, and S. J. Shin, "Development of a CNN-based real-time monitoring algorithm for additively manufactured molybdenum," *Sensors and Actuators A: Physical*, vol. 352, p. 114205, 2023.

[20] B. Ku, K. Kim, and J. Jeong, "Real-time ISR-YOLOv4 based small object detection for safe shop floor in smart factories," *Electronics*, vol. 11, no. 15, p. 2348, 2022.

[21] P. Lou, J. Li, Y. Zeng, B. Chen, and X. Zhang, "Real-time monitoring for manual operations with machine vision in smart manufacturing," *Journal of Manufacturing Systems*, vol. 65, pp. 709–719, 2022.

[22] S. Ma *et al.*, "Edge–cloud cooperation-driven sustainable smart optimization strategy for additive manufacturing," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2026.

[23] D. Nimma *et al.*, "Object detection in real-time video surveillance using attention-based transformer-YOLOv8 model," *Alexandria Engineering Journal*, vol. 118, pp. 482–495, 2025.

[24] A. S. Ozer and I. Cinar, "Real-time and fully automated robotic stacking system with deep learning-based visual perception," *Sensors*, vol. 25, no. 22, p. 6960, 2025.

[25] M. Park and J. Jeong, "Design and implementation of machine vision-based quality inspection system in mask manufacturing process," *Sustainability*, vol. 14, no. 10, p. 6009, 2022.

[26] S. Prayogi, "Implementation of CNN-based computer vision for personal protective equipment detection in the oil and gas industry," *Jurnal Sisfokom (Sistem Informasi dan Komputer)*, vol. 14, no. 4, pp. 454–460, 2025.

[27] K. N. Rahman, S. C. Banik, R. Islam, and A. Al Fahim, "A real-time monitoring system for accurate plant leaves disease detection using deep learning," *Crop Design*, vol. 4, no. 1, p. 100092, 2025.

[28] K. Rajendra, "Applications of vision inspection systems in manufacturing industry," 2024. [Online]. Available: https://kritikalsolutions.com/vision-inspection-systems-in-manufacturing-industry/. [Accessed: Mar. 6, 2026].

[29] N. Rane, "YOLO and Faster R-CNN object detection for smart Industry 4.0 and Industry 5.0: Applications, challenges, and opportunities," *SSRN*, 2023.
[30] M. Ryalat, H. ElMoaqet, and M. AlFaouri, "Design of a smart factory based on cyber-physical systems and Internet of Things towards Industry 4.0," *Applied Sciences*, vol. 13, no. 4, p. 2156, 2023.
[31] M. Ryalat, E. Franco, H. Elmoaqet, N. Almtireen, and G. Al-Refai, "The integration of advanced mechatronic systems into Industry 4.0 for smart manufacturing," *Sustainability*, vol. 16, no. 19, p. 8504, 2024.
[32] T. Schlosser, M. Friedrich, F. Beuth, and D. Kowerko, "Improving automated visual fault inspection for semiconductor manufacturing using a hybrid multistage system of deep neural networks," *Journal of Intelligent Manufacturing*, vol. 33, no. 4, pp. 1099–1123, 2022.
[33] I. Shafi *et al.*, "Deep learning-based real-time defect detection for optimization of aircraft manufacturing and control performance," *Drones*, vol. 7, no. 1, p. 31, 2023.
[34] M. Shahin *et al.*, "A novel fully convolutional neural network approach for detection and classification of attacks on industrial IoT devices in smart manufacturing systems," *The International Journal of Advanced Manufacturing Technology*, vol. 123, no. 5, pp. 2017–2029, 2022.
[35] M. Shaloo, G. Princz, R. Hörbe, and S. Erol, "Flexible automation of quality inspection in parts assembly using CNN-based machine learning," *Procedia Computer Science*, vol. 232, pp. 2921–2932, 2024.
[36] A. Singh, "Deep dive into the world of CNNs," 2023. [Online]. Available: https://ai.plainenglish.io/deep-dive-into-the-world-of-cnns-8cf22cd84e7. [Accessed: Mar. 6, 2026].
[37] H. Tyagi, V. Kumar, and G. Kumar, "A review paper on real-time video analysis in dense environment for surveillance system," in *Proc. 2022 Int. Conf. Fourth Industrial Revolution Based Technology and Practices (ICFIRTP)*, 2022, pp. 171–183.
[38] T. Wang, P. Zheng, S. Li, and L. Wang, "Multimodal human–robot interaction for human-centric smart manufacturing: A survey," *Advanced Intelligent Systems*, vol. 6, no. 3, p. 2300359, 2024.
[39] C. Xia, Z. Pan, Y. Li, J. Chen, and H. Li, "Vision-based melt pool monitoring for wire-arc additive manufacturing using deep learning method," *The International Journal of Advanced Manufacturing Technology*, vol. 120, no. 1, pp. 551–562, 2022.
[40] J. Xie, A. Saluja, A. Rahimizadeh, and K. Fayazbakhsh, "Development of automated feature extraction and convolutional neural network optimization for real-time warping monitoring in 3D printing," *International Journal of Computer Integrated Manufacturing*, vol. 35, no. 8, pp. 813–830, 2022.
[41] J. Xu *et al.*, "A review on AI for smart manufacturing: Deep learning challenges and solutions," *Applied Sciences*, vol. 12, no. 16, p. 8239, 2022.
[42] W. Yan, J. Wang, S. Lu, M. Zhou, and X. Peng, "A review of real-time fault diagnosis methods for industrial smart manufacturing," *Processes*, vol. 11, no. 2, p. 369, 2023.
[43] I. Yousif, L. Burns, F. El Kalach, and R. Harik, "Leveraging computer vision towards high-efficiency autonomous industrial facilities," *Journal of Intelligent Manufacturing*, vol. 36, no. 5, pp. 2983–3008, 2025.
[44] N. Zendehdel, H. Chen, and M. C. Leu, "Real-time tool detection in smart manufacturing using You-Only-Look-Once (YOLO) v5," *Manufacturing Letters*, vol. 35, pp. 1052–1059